# SMART INDIA HACKATHON 2026

Problem Statement 26261 | AICTE Regulation Bureau

# NirikshAI

*AI-Powered Application Validation, Document Authentication and Regulatory Compliance Verification*

---

*"Check early. Prove everything. Humans decide."*

**SIH 2026 — Problem Statement 26261**

AICTE Regulation Bureau, Ministry of Education, Government of India

Category: Software | Theme: Smart Automation

# SMART INDIA HACKATHON 2026

Problem Statement 26261 | AICTE Regulation Bureau

This document is a planning and design report prepared for the Smart India Hackathon 2026 (SIH 2026). It describes proposed system architecture, features and evaluation plans. All code, metrics and prototypes described are plans as of 20 October 2026 and have not been deployed or tested against real AICTE data. Synthetic documents used in the prototype are clearly labelled "SYNTHETIC" and are not real institutional documents. All rule-clause references are illustrative placeholders; the actual AICTE Approval Process Handbook 2024-27 must be read to fill in real clause numbers.

# 0. How to Read This Report

This report covers SIH 2026 Problem Statement **26261: AI-Powered Application Validation, Document Authentication and Regulatory Compliance Verification** (Organization: AICTE, Regulation Bureau; Category: Software; Theme: Smart Automation). Prepared on 20 October 2026 for a submission deadline of 28 October 2026.

Every important statement is tagged so you can tell evidence from opinion:

| Tag | Meaning |
|---|---|
| [Official PS] | Taken directly from the problem statement text supplied. |
| [Verified] | Found in a source retrieved today. Secondary sources are marked as such. |
| [Inference] | Reasoned conclusion from verified facts. Not confirmed by AICTE. |
| [Assumption] | A design choice or planning figure. Change if the team knows better. |
| [Verify] | Something the team must check before relying on it. |

**Honest limits.** Three things NOT done: (1) The AICTE Approval Process Handbook 2024-27 was not read — every threshold in the example rules is a placeholder; downloading the Handbook and extracting the real checklist is a Day-1 task. (2) The official SIH portal was not opened — confirm the deadline time, slide limit, video requirement and number of statements allowed per team. (3) Nothing has been built or tested yet: all code and metrics are plans.

# 1. Executive Summary and Recommendation

## 1.1 Recommendation

**Continue with PS 26261.** On its own merits it is a strong choice for a software team, for four reasons:

- **The whole problem can be demonstrated without any restricted access.** The statement itself requires demos on "authorized, anonymized or synthetic datasets" [Official PS].
- **It maps onto what you already build.** Document upload, role-based dashboards, audit trails and rules are standard full-stack work in Next.js, Supabase and Clerk. Only the document-forensics service needs Python.
- **It rewards rigor over flash.** The statement stresses explainability, evidence links, advisory-only AI and human review.
- **It has natural, measurable evaluation metrics** (extraction accuracy, defect-detection recall, verification states, latency) that can be computed on a labelled synthetic set.

## 1.2 What we propose to build: NirikshAI

**NirikshAI** ("nirikshan" = inspection) is a decision-support layer that sits beside the AICTE application portal. When an institution submits an Extension of Approval (EoA) application, NirikshAI (a) classifies the application category and loads the right checklist, (b) reads every uploaded document with OCR and extracts its fields, (c) compares those fields with each other and with institutional records already held by AICTE, (d) triages documents for tampering and duplication, (e) verifies documents with issuing authorities through pluggable adapters and records one of seven explicit statuses, (f) flags infrastructure-evidence gaps, and (g) writes everything to a tamper-evident audit trail. A scrutiny officer sees a ranked, evidence-linked list of findings and makes every decision.

**Tagline:** Check early. Prove everything. Humans decide.

## 1.3 Key innovations

| # | Innovation | Why it matters |
|---|---|---|
| 1 | Evidence-linked findings: every flag points to the page and bounding box that caused it | Meets the "explainable, evidence-supported" safeguard; lets an officer confirm a finding in seconds |
| 2 | Rules as versioned data, not code: checklists per application category, each rule citing its Handbook clause | Meets "all checks must be configurable"; the Handbook changes every cycle |
| 3 | Issuer-adapter framework with seven verification states from the PS | Real-world issuers differ; one interface handles all and degrades safely |
| 4 | Tamper triage that outputs risk indicators, never verdicts, using layered signals | Matches "preliminary indicators requiring human verification" |
| 5 | Hash-chained audit log of rules applied, records read, source responses and officer actions | Gives accountability without needing blockchain |

## 1.4 Three decisions the team must make today

1. **Confirm the tech stack in Section 9** (Next.js + Supabase + Clerk + a Python FastAPI AI service; Azure used only for hosting and optional OCR). Changing it later costs more days than the team has.
2. **Confirm what the deadline requires** on the SIH portal: slides only, or slides plus video plus prototype link.

3. **Assign six roles** (Section 15). SIH teams are exactly six members with at least one woman [Verified, secondary sources].

# 2. SIH 2026 Context and Submission Facts

| Fact | Status | Source / note |
|---|---|---|
| SIH 2026 is run by the Ministry of Education's Innovation Cell (MIC) and AICTE | [Verified, secondary] | whereuelevate.com, blogs.reskilll.com |
| Team = exactly 6 students from the same institute, at least 1 female | [Verified, secondary] | blogs.reskilll.com; bcrec.ac.in |
| Idea submission is a PPT of at most 6 slides, including the title slide | [Verified, secondary] | Template copies on Google Slides/SlideShare; bcrec.ac.in |
| After shortlisting, a 36-hour Grand Finale at nodal centres, December 2026 | [Verified, secondary] | blogs.reskilll.com; bcrec.ac.in |
| Online evaluation considers novelty, complexity, clarity, feasibility, impact, UX | [Verified, secondary] | MIET page on internshala |
| Submission deadline 28 October 2026; whether a video or prototype link is required | [Verify] | Deadline from user's message only. Confirm on the portal. |

**Planning consequence.** Whatever the portal asks for, the idea PPT is the first filter. A reviewer reads six slides, possibly watches a short video, and may click a link. So the plan builds a thin but real end-to-end prototype AND a very tight 6-slide deck in parallel.

# 3. Problem Statement Interpretation

## 3.1 In simple language

When a college applies to AICTE each year to continue (extend) its approval, it uploads many documents: affidavits, NOCs from government or university, accreditation certificates, fee and payment proofs, faculty lists, infrastructure photographs and more. Officers must check that nothing is missing or expired, that numbers agree across documents and with what AICTE already knows, that documents are genuine, and that the infrastructure claimed is plausible.

The ask: build an AI-assisted platform that does the **first pass** of this scrutiny automatically, shows its evidence, and routes the doubtful cases to humans. It must **never** make the final decision or accuse anyone of forgery.

## 3.2 The exact problem to solve

[Inference] The core problem is not "detect forgery". It is **early, consistent, evidence-backed triage**: tell officers (and institutions) as early as possible which applications are complete and consistent, which have fixable gaps, and which need a human to look closer — and prove why.

## 3.3 Explicit requirements

| ID | Requirement [Official PS] | Where we address it |
|---|---|---|
| R1 | Identify application category and applicable checklist | Module M3 (category + rule set) |
| R2 | Retrieve relevant records from authorized AICTE systems (affidavits, accreditation, NOCs, previous EoA, approved programmes, faculty records, TER payment status) | Module M4 (records adapter, mock in prototype) |
| R3 | OCR, classification and data extraction to find missing fields, expired documents, inconsistencies | Modules M2, M3 |
| R4 | Generate a structured validation report; allow rectification where permitted | Modules M3, M8 (report + resubmit loop) |
| R5 | Detect suspicious alterations, duplicates, inconsistent metadata; each flag shows evidence; flags are preliminary only | Module M5 |
| R6 | Verify via issuing authorities; record source, timestamp, result; 6 statuses; route inconclusive to manual confirmation | Module M6 + state machine |
| R7 | Compare against configurable checklists from Handbook; computer-vision infrastructure checks; advisory only | Modules M3, M7 |
| R8 | Dashboard: status, deficiencies, verification results, cases needing intervention | Module M8 |
| R9 | Alerts for missing info, expired docs, pending verification, anomalies | Module M9 |
| R10 | Audit trail of rules applied, records checked, source responses, actions | Module M10 |
| R11 | Role-based access, secure document handling, human review before adverse action | Cross-cutting (Section 11) |
| R12 | Prototype integrated with, or designed for integration with, the AICTE portal | Integration contract + mock records API |
| R13 | Demonstrate on authorized, anonymized or synthetic data; evaluate metrics | Section 12 (evaluation plan) |

## 3.4 Essential safeguards and design responses

| Safeguard | Design response |
|---|---|
| Checks configurable per Handbook, regulations, instructions | Rules stored as versioned data with source clause; admin UI edits and versions them |
| No bypassing access controls; no assumption of restricted database access | All external access through adapters needing credentials; prototype uses clearly-labelled mocks |
| Protect institutional and personal data; logging; retention | Private storage, RLS, signed URLs, access log, retention timer, field-level masking |
| Flags explainable and evidence-supported | Finding schema requires evidence (document, page, bbox, extracted value, rule id) |
| AI output is decision support only; humans decide | No automated rejection state exists in the data model |

## 3.5 Stakeholders and personas

| Persona | Needs | What they see in NirikshAI |
|---|---|---|
| Institution registrar / compliance officer (primary submitter) | Know about problems before submitting; avoid deficiency rounds | Pre-submission checklist, completeness score, fix-it list with exact document and field |
| Scrutiny officer, Regulation Bureau (primary reviewer) | Spend time only where judgement is needed | Ranked work queue, evidence viewer, per-finding confirm/dismiss, verification status |
| Senior officer / regional office | Consistency across officers; escalations | Dashboard by region/category, unresolved items, workload |
| Admin / policy cell | Update rules when the Handbook changes | Rule manager with versions and clause references |
| Auditor / Vigilance | Reconstruct why something happened | Read-only audit trail with hash-chain integrity check |

# 4. Current State, Existing Solutions and the Gap

## 4.1 How the process works today

- **EoA letters are computer-generated.** Public EoA letters for 2026-27 carry numbers of the form "Region/1-<11 digits>/2026/EOA", cite the Approval Process Handbook 2024-25 to 2027, and list approved programmes with intake [Verified: institution-published copies, e.g., cvru.ac.in, nbs.edu.in]. Cross-checking "letter number, institution ID, programme, intake" against AICTE records is a concrete, demonstrable validation.
- **Applications need supporting state/university documents.** A Kerala DTE circular lists what institutions must submit for EoA: a copy of the AICTE application, the latest AICTE approval and university affiliation, and NOCs for extension of existing courses [Verified: dtekerala.gov.in].
- **Verification channels vary.** DigiLocker is a consent-based document exchange with a PKCE OAuth2 flow and HMAC-signed PDF/XML responses [Verified: api.apisetu.gov.in; secondary: didit.me]. Real DigiLocker access cannot be obtained in eight days, so the prototype uses an adapter with a mock.
- **Commercial wrappers exist** (Gridlines, HyperVerge, Signzy) but target KYC for banks, not AICTE institutional documents — they do not solve this problem out of the box [Inference].
- **How AICTE officers scrutinise today** is not publicly documented. Do not claim that no AICTE automation exists; say "we found no public description of AI-assisted pre-scrutiny in the AICTE approval workflow" [Verify].

## 4.2 Comparable solutions

| Solution | What it does well | Limits for this PS | Our stance |
|---|---|---|---|
| DigiLocker (NeGD) | Consented, issuer-signed documents; national reach; API for registered requesters | Only for issuers on the platform; registration needed; many institutional certificates not present | Reuse as the adapter model; mock in prototype |
| Commercial verification APIs (Gridlines, HyperVerge, Signzy) | Fast integration, support teams | Paid; KYC-oriented; not AICTE-specific; data leaves AICTE boundary | Avoid as a dependency; possible future adapter |
| Generic IDP services (Azure AI Doc Intelligence, AWS Textract) | Strong OCR, layout, tables, prebuilt models | Do not know AICTE rules; cloud data-residency; free-tier limits | Optional OCR engine behind an interface; open-source fallback |
| Open-source OCR (PaddleOCR, Tesseract) + PDF tools (PyMuPDF, pikepdf) | Free, self-hostable, no data leaves the server | Need engineering; accuracy varies on poor scans | Core of the prototype |

## 4.3 Root-cause analysis

**Figure 1 – NirikshAI Root-Cause Analysis Tree**

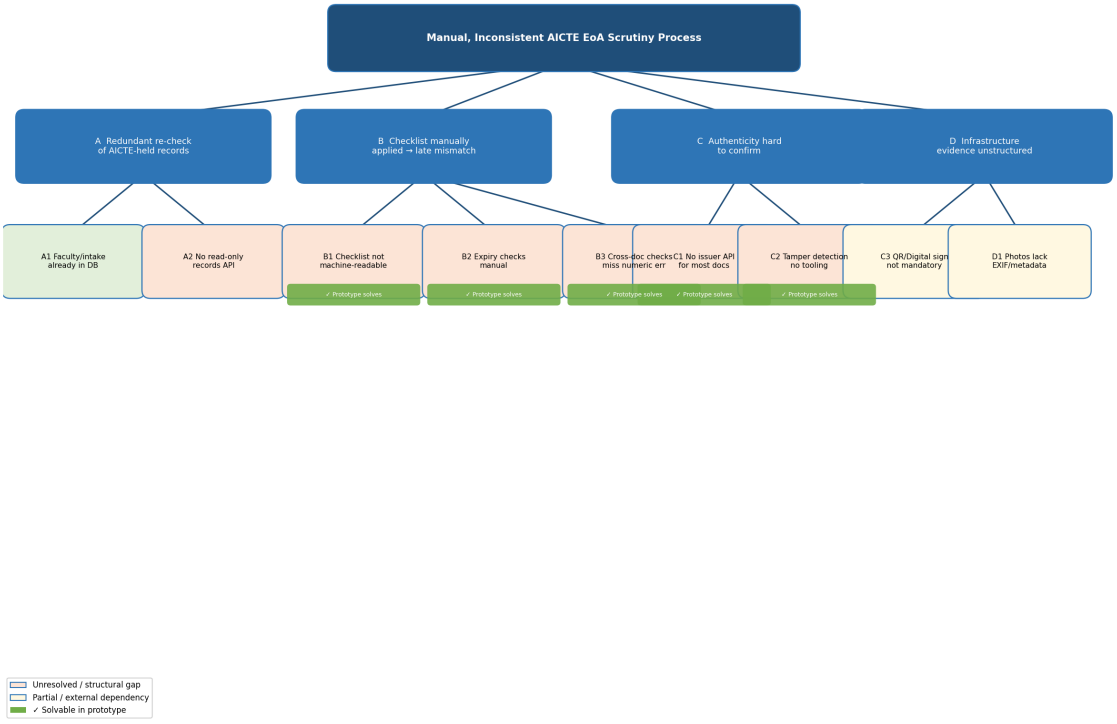



*Figure 1: Root-cause tree. Green = what is realistic to solve in an SIH prototype.*

# 5. Problem-Statement Scoring Framework

Scores are 1 (poor) to 5 (excellent). Weights sum to 100.

| # | Criterion | Weight | Score | Reasoning |
|---|---|---|---|---|
| 1 | Alignment with official PS | 8 | 5 | Very detailed PS with explicit deliverables |
| 2 | Severity / importance of problem | 7 | 4 | Real operational burden for a national regulator |
| 3 | Evidence of unmet need | 6 | 3 | PS itself states it; no independent public data found |
| 4 | Innovation potential | 8 | 4 | Evidence-linked, rules-as-data, fail-safe verification |
| 5 | Technical feasibility in 8 days | 12 | 4 | Standard components; forensics and OCR are the hard parts |
| 6 | Data / API / resource availability | 8 | 3 | Synthetic data is allowed; real issuers and AICTE records are not |
| 7 | Demonstrable end-to-end prototype | 10 | 5 | Clear flow: upload, findings, verification, officer decision |
| 8 | Measurable impact | 6 | 4 | Metrics defined in PS; can be computed on synthetic set |
| 9 | Scalability / deployment potential | 5 | 4 | Sidecar design, stateless workers |
| 10 | Security, privacy, compliance fit | 6 | 4 | PS demands them; controls are showable |
| 11 | Differentiation from existing solutions | 6 | 3 | Components common; integration is the edge |
| 12 | Withstands technical questioning | 6 | 4 | Strong if the team stays honest about limits |
| 13 | Low dependency / execution risk | 8 | 4 | Mocks remove external dependency; OCR quality is the main risk |
| 14 | Fit with team skills | 4 | 4 | Good, assuming one member can do Python |

**Weighted score for PS 26261: approximately 3.95 / 5 (79 / 100)** [Assumption: scoring based on analysis above].

# 6. Solution Design: NirikshAI

## 6.1 Product definition

| Item | Definition |
|---|---|
| Name / tagline | NirikshAI: Check early. Prove everything. Humans decide. |
| Value proposition | Turns the first pass of EoA scrutiny from hours of manual cross-checking into a ranked, evidence-linked list of findings that an officer can confirm or dismiss in seconds, with every step auditable. |
| Users | Institution compliance officers; scrutiny officers; senior officers; admins; auditors |
| Main use cases | (1) Pre-submission self-check. (2) Officer first-pass triage. (3) Issuer verification tracking. (4) Infrastructure-evidence advisory. (5) Rule updates when the Handbook changes. (6) Audit review. |

## 6.2 Five key innovations

### Innovation 1: Evidence-linked findings

| Aspect | Detail |
|---|---|
| Limitation addressed | Black-box scores make officers re-do the work to trust them |
| How it works | Every finding stores: rule id, Handbook clause, document id, page, bounding box, extracted value, comparison value, confidence. The viewer opens the page and highlights the region. |
| Why different | Findings are claims that can be checked, not a risk number. |
| Success measure | % of findings with a resolvable evidence link (target 100%); median officer time to confirm a finding. |

### Innovation 2: Rules as versioned data

| Aspect | Detail |
|---|---|
| Limitation addressed | Hard-coded checks go stale when the Handbook changes |
| How it works | Each category has a rule set (JSON/YAML) with typed rules: required_document, field_present, date_valid_for_year, cross_field_equal, numeric_leq, fuzzy_match, status_equals, identifier_format. |
| Why different | Policy cell, not developers, owns the checklist. |
| Success measure | Time to add a new rule through the UI (minutes, demonstrated live); 100% of findings carry rule version. |

### Innovation 3: Seven-state verification with fail-safe routing

| Aspect | Detail |
|---|---|
| Limitation addressed | Issuers differ; a binary "verified / not verified" hides why |
| How it works | One adapter interface: verify(document, extracted_fields) returns {status, source, timestamp, comparison}. States: verified, mismatch_detected, expired, verification_pending, source_unavailable, manual_review_required. Timeouts retry with back-off, then route to manual review. |

| Aspect | Detail |
|---|---|
| Why different | Unavailable source never becomes a silent pass or a false accusation. |
| Success measure | % of documents reaching a conclusive state; % correctly routed to manual review when the mock source fails. |

### Innovation 4: Tamper triage with calibrated language

| Aspect | Detail |
|---|---|
| Limitation addressed | Forgery "detectors" overclaim; false accusations harm institutions |
| How it works | Layered, individually explainable signals: PDF incremental updates, creation/modification date gaps, font anomalies, text layer vs OCR mismatch, perceptual duplicate hashes, identifier format checks. Output is "review" or "priority review" labels. The word "forged" never appears. |
| Implementation | PyMuPDF, pikepdf, OpenCV/PIL, imagehash, pyHanko; all open source |
| Success measure | Recall and precision per indicator on a labelled synthetic set; report false-positive rate on clean documents. |

### Innovation 5: Hash-chained audit log

| Aspect | Detail |
|---|---|
| Limitation addressed | Regulatory decisions must be reconstructable and tamper-evident |
| How it works | hash = SHA-256(prev_hash + canonical JSON of the event). A "verify chain" button recomputes and reports the first broken link. |
| Why different | Gives the integrity benefit people want from blockchain with a normal database. |
| Success measure | Chain verification passes; a deliberate DB edit is detected in the demo. |

## 6.3 What we deliberately leave out

- **Blockchain** (a hash-chained table achieves the stated goal).
- **Training a custom forgery CNN** (no labelled real data; would produce fake-looking accuracy numbers).
- **Fully automatic pass/fail decisions** (violates the PS safeguards).
- **Microservice sprawl**: two deployable units only (Next.js app, Python AI service).
- **LLM-as-judge for compliance** — LLMs may help extract fields; rule decisions stay deterministic.

# 7. End-to-End User Journey, Edge Cases and Recovery

## 7.1 Normal flow

1. **Institution signs in** (Clerk) and starts an EoA application for 2026-27. The system identifies the category and displays the checklist.
2. **Uploads documents.** Each file is hashed (SHA-256), stored in a private bucket, and queued.
3. **Pre-scrutiny runs** (seconds to a couple of minutes): classify, OCR/extract, run rules, run cross-checks against institutional records, run tamper triage, start issuer verification.
4. **Institution sees a validation report**: missing documents, expired documents, field inconsistencies with the exact place, and what to fix. It uploads corrected documents; the pipeline re-runs on changed files only.
5. **Application is submitted.** Officer queue is ordered by priority (blocking gaps, verification pending, tamper indicators, infrastructure flags).
6. **Officer opens a case**, reviews findings with evidence, confirms or dismisses each with a reason, requests manual confirmation from an issuer where needed, and records a decision.
7. **Alerts** go out for unresolved items; everything lands in the audit trail.

## 7.2 Failure and edge cases

| Situation | System behaviour | Recovery |
|---|---|---|
| Document is a poor scan or unreadable | Field confidence below threshold; finding "Unreadable: re-upload", not "missing" | Institution re-uploads; officer can override |
| Unknown document type | Classifier returns "unclassified"; goes to manual classification | Officer tags; tag becomes training example (future) |
| Document expired for the academic year | Status EXPIRED with dates and evidence | Upload renewal or officer waives with reason |
| Same value differs between two documents | Inconsistency finding showing both values and both locations | Institution corrects the wrong document |
| Issuer source down or times out | Status SOURCE_UNAVAILABLE, retry with back-off, then MANUAL_REVIEW_REQUIRED | Officer confirms manually; result recorded |
| Malicious file (malware, zip bomb, embedded script) | Type and size checks, page limit, sandboxed parsing with timeout, PDF JavaScript stripped | File rejected with reason; event logged |
| Duplicate document in another institution's application | Priority-review indicator with both references (visible to officers only) | Officer investigates |
| OCR / AI service unavailable | Application accepted and queued; status "Analysis pending"; deterministic checks still run | Worker retries; officer sees banner |

# 8. Feature Prioritisation (P0 / P1 / P2)

| Priority | Feature | Why / drop rule |
|---|---|---|
| P0 | Auth + roles (institution, officer, admin); application + document upload to private storage | Without these there is no demo |
| P0 | Category checklist (1 category fully: EoA existing institution) with completeness and expiry checks | Core value; PS R1, R3 |
| P0 | OCR + field extraction for 5 document types on synthetic documents (EoA letter, affidavit, NOC, accreditation certificate, fee/TER proof) | Core; keep to native PDFs first, scans second |
| P0 | Rule engine with 6-8 rule types and cross-document consistency | Core differentiator |
| P0 | Findings with evidence link and officer confirm/dismiss | Core differentiator |
| P0 | Verification adapters: mock issuer + QR/reference parser; seven states; manual queue | PS R6 |
| P0 | Dashboard (status, deficiencies, pending) + audit log | PS R8, R10 |
| P1 | Tamper triage (PDF structure, metadata, duplicates, ELA) | PS R5; cut ELA first if short of time, keep PDF/metadata/duplicate |
| P1 | Email/in-app alerts | PS R9; in-app only is acceptable |
| P1 | Rule manager UI with versioning | Shows configurability live |
| P1 | Infrastructure-evidence advisory (EXIF, duplicate photos, area cross-check) | PS R7; honest "advisory" label |
| P1 | Metrics page reporting evaluation results | Makes claims checkable |
| P2 | Hindi/Devanagari OCR; multi-category rule sets; real DigiLocker adapter; video analysis; regional analytics | Future scope slide |

# 9. Recommended Technology Stack

| Layer | Choice | Essential? |
|---|---|---|
| Web app (all UIs + API) | **Next.js (App Router) + TypeScript**, Tailwind + shadcn/ui, hosted on Vercel | Yes |
| Authentication / roles | **Clerk** (organisations or public metadata for roles: institution, officer, admin, auditor) | Yes |
| Database + files | **Supabase**: PostgreSQL with Row Level Security, private Storage buckets, Realtime for live status | Yes |
| Job queue | Postgres table + worker polling (or Supabase Queues/pgmq) [Verify] | Yes (keep simple) |
| Document AI service | **Python 3.11 + FastAPI** with PyMuPDF, pikepdf, pyHanko, OpenCV, imagehash, PaddleOCR/Tesseract | Yes |
| OCR (optional upgrade) | **Azure AI Document Intelligence** behind an interface; open-source OCR is the default | No (nice to have) |
| Field extraction help | Regex + layout rules first; optional LLM for messy fields, with output schema validation | No (fallback exists) |
| Hosting Python service | Azure Container Apps or Render/Fly.io (Docker) [Verify free limits] | Yes |
| PDF viewer with highlights | pdf.js (react-pdf) | Yes |
| Charts | Recharts | Yes |
| Testing | Vitest (rules), pytest (AI service), Playwright (one end-to-end test) | Yes (minimal) |
| CI/CD | GitHub Actions + Vercel previews | Optional |

**Critical-dependency rule.** Anything the demo cannot work without must run on free, accessible infrastructure the team controls. So: open-source OCR is the default engine, the mock issuer and mock AICTE-records API are part of your own deployment, and Azure/LLM calls are optional upgrades behind feature flags. If Azure credits are not ready by Day 3, drop Azure entirely.

## 9.3 Fallbacks for critical dependencies

| Dependency | Failure | Fallback |
|---|---|---|
| Supabase free tier | Project paused / storage full | Un-pause the day before; keep a local Docker Postgres + MinIO profile as backup |
| Clerk | Outage / rate limit | Demo mode with three seeded users behind an env flag (clearly labelled), or Supabase Auth |
| Python AI service host | Free instance sleeps or cold starts | Keep-alive ping; run the same container locally on a laptop for the live demo |
| Azure OCR / any LLM | Quota or network failure | Default PaddleOCR/Tesseract + regex extraction; cached results for the demo set |
| Internet at the venue | Down | Pre-recorded video of the full flow plus local docker-compose demo |

# 10. Technical Architecture

## 10.1 System architecture

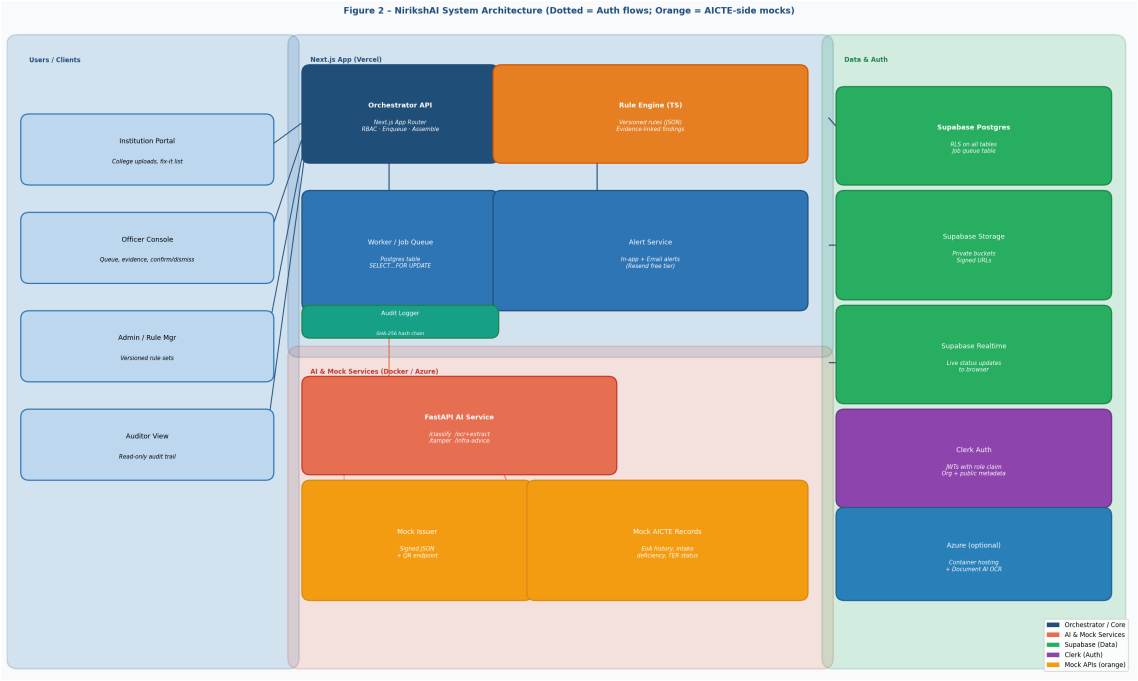



*Figure 2: System architecture. Dotted lines = auth. Orange = AICTE-side records (mock in the prototype).*

## 10.2 Components

| Component | Responsibility | Essential? |
|---|---|---|
| Institution Portal | Create application, upload, see validation report, rectify | Yes |
| Officer Console | Queue, case view, evidence viewer, confirm/dismiss, request manual confirmation, decision | Yes |
| Admin / Rule Manager | Edit rule sets and thresholds with versions | P1 (JSON files are the fallback) |
| Orchestrator API (Next.js) | RBAC, input validation, enqueue jobs, assemble reports, call adapters | Yes |
| Rule Engine (TypeScript) | Evaluate typed rules against extracted fields and records; emit findings | Yes |
| Audit Logger | Append-only, hash-chained events | Yes |
| AI Service (FastAPI) | /ocr, /classify, /extract, /tamper, /infra-advice; stateless; signed requests | Yes |
| Verification Adapters | Uniform verify() interface over mock issuer, QR/reference parser, signature validator, manual queue | Yes |
| Records Adapter | Read-only calls to AICTE institutional records (mock REST in prototype) | Yes |
| Supabase Postgres + Storage | System of record; RLS; private files | Yes |

## 10.3 Main workflow sequence

```
Institution -> Web: upload document(s)
Web -> Storage: put file (private), compute sha256
Web -> DB: insert document(status=queued); audit(DOC_UPLOADED)
Worker -> DB: claim job (SELECT ... FOR UPDATE SKIP LOCKED)
Worker -> AI: POST /classify {file_url}      -> doc_type, confidence
Worker -> AI: POST /ocr+extract {file_url}   -> fields[] with page + bbox + confidence
Worker -> AI: POST /tamper {file_url}        -> indicators[] with reasons + evidence
Worker -> Records: GET /institutions/{id}/records -> official values
Worker -> RuleEngine: evaluate(rule_set vN, fields, records) -> findings[]
Worker -> Adapters: verify(doc, fields)      -> status in 7 states (retry/backoff)
Worker -> DB: save fields, findings, verifications; audit(RULES_APPLIED)
DB -> Web (Realtime): application status updated
Alert Service: notify on missing/expired/pending/anomaly
Officer -> Web: open case -> confirm/dismiss -> audit(OFFICER_ACTION)
```

## 10.4 Verification state machine

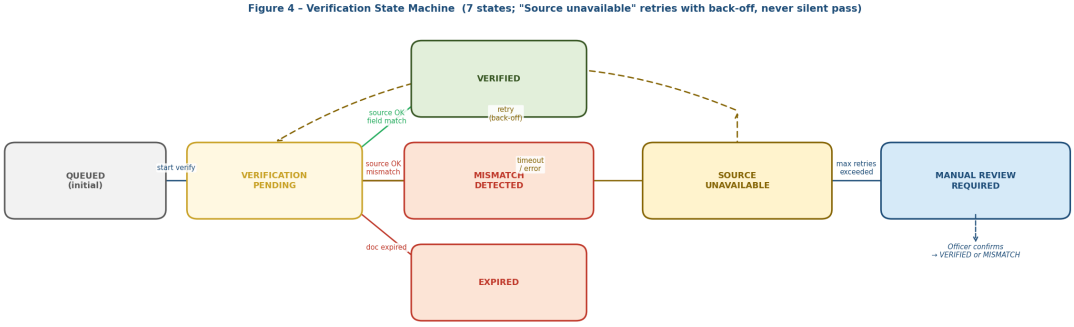



*Figure 4: Verification states. "Source unavailable" retries with back-off then routes to manual review; nothing ends as a silent pass.*

## 10.5 Data model

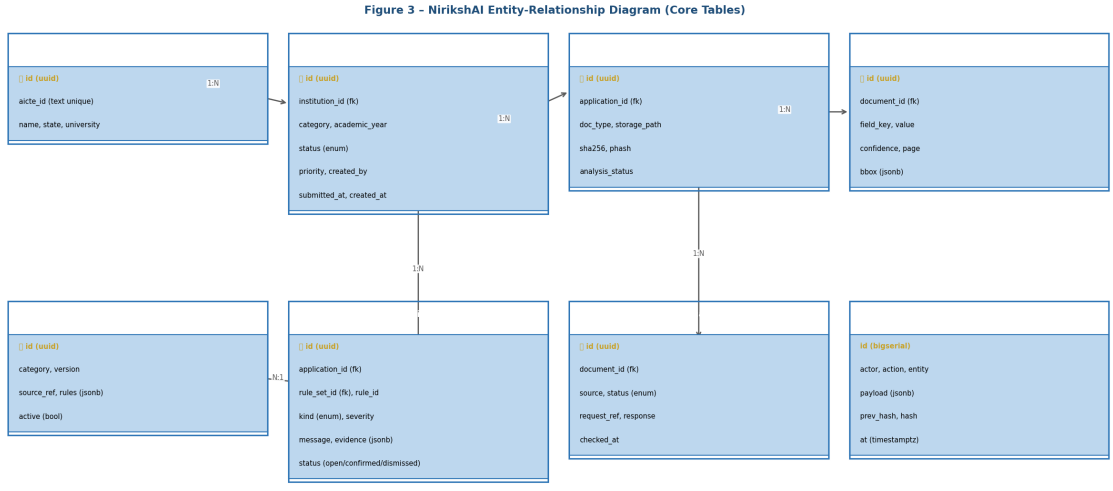



*Figure 3: Entity-relationship diagram (core tables).*

## 10.6 API specification (core endpoints)

| Method + path | Role | Purpose | Key request / response |
|---|---|---|---|
| POST /api/applications | institution | Create application | in: {category, academic_year}; out: {id, checklist[]} |
| POST /api/applications/:id/documents | institution | Register uploaded file | in: {path, filename}; out: {document_id, status:"queued"} |
| GET /api/applications/:id/report | institution, officer | Validation report | out: {completeness, findings[], verifications[]} |
| POST /api/applications/:id/submit | institution | Submit application | out: {status} |
| GET /api/queue | officer | Prioritised cases | query: region, category, status |
| POST /api/findings/:id/review | officer | Confirm or dismiss | in: {decision, notes} |
| POST /api/documents/:id/verify | officer | Trigger / retry verification | out: {status} |
| POST /api/documents/:id/manual-confirm | officer | Record manual issuer confirmation | in: {method, reference, outcome, notes} |
| GET/PUT /api/rule-sets/:category | admin | Read / publish new version | in: {rules[], source_ref} |
| GET /api/audit?entity= | officer, auditor | Audit trail | out: events[] + chain_ok |
| GET /api/audit/verify-chain | auditor | Recompute chain | out: {ok, first_broken_id?} |

## 10.7 Repository layout

```
nirikshai/
  apps/web/                # Next.js (App Router, TS)
    app/(institution)/  app/(officer)/  app/(admin)/  app/api/
    lib/rules/             # evaluator + rule types + tests
    lib/audit.ts  lib/adapters/  lib/supabase.ts
  services/ai/             # FastAPI
    main.py  ocr.py  classify.py  extract.py  tamper/  infra.py  tests/
  services/mocks/          # mock issuer + mock AICTE records
  data/synthetic/          # generated documents + labels.csv (clearly marked SYNTHETIC)
  rules/                   # EOA_EXISTING.v1.json ...
  db/                      # schema.sql, seed.sql, rls.sql
  eval/                    # scripts that compute metrics on data/synthetic
  docs/                    # architecture, PPT, demo script
  docker-compose.yml       # offline fallback: postgres, minio, ai, mocks
```

# 11. Security, Privacy and Responsible-AI Controls

| Area | Control in the design |
|---|---|
| Authentication | Clerk sessions; JWT with role claim verified on every API call and in Supabase RLS policies |
| Authorisation | Roles: institution (own applications only), officer (assigned region/category), admin (rules), auditor (read-only logs). RLS on every table; service-role key never reaches the browser |
| Document handling | Private buckets; short-lived signed URLs; file-type, size and page-count limits; parsing in an isolated container with timeout; strip active content from PDFs; optional antivirus scan hook |
| Data protection | TLS in transit; provider encryption at rest; mask personal identifiers in UI and logs; no raw documents in logs |
| Access logging | Every read of a document or record is an audit event with actor and purpose |
| Retention | Configurable retention per data class; scheduled purge job (prototype: documented, one demo job) |
| Secrets | Environment variables only; rotate keys after the hackathon; no keys in git |
| Responsible AI | Indicators not verdicts; evidence on every flag; dismissal feedback tracked; false-positive rate published; no automated adverse action; human confirmation recorded |
| Prompt-injection (if LLM used) | Treat document text as untrusted data; LLM output must pass a JSON schema; LLM never changes a rule outcome |
| Threats named in Q&A; | Malicious upload, privilege escalation, IDOR, log tampering, adversarial documents designed to fool OCR; each has a control above |

# 12. Data, AI/ML and Evaluation Plan

## 12.1 Data strategy

**No real AICTE application data is available, and we must not scrape or use real institutions' documents without authorisation.** The PS allows "authorized, anonymized or synthetic datasets". We therefore generate a **clearly labelled synthetic corpus**.

- **Templates:** 5 document templates modelled on public EoA letters and common supporting documents (affidavit, NOC, accreditation certificate, fee/TER receipt, faculty list). Fictional institution names and IDs. Every file marked "SYNTHETIC" in a footer and in metadata.
- **Variants:** approximately 150-200 documents: clean (~60%), with a missing field, expired date, cross-document mismatch, edited number/date (injected tamper), duplicate across two applications, low-quality scan (blur, skew, noise, JPEG artefacts).
- **Labels:** a CSV of ground truth per document: type, fields, injected defect type and location.

## 12.2 Models and methods

| Task | Baseline | Upgrade (only if time) | Metric |
|---|---|---|---|
| Classification | Keyword + layout rules on first-page text | TF-IDF + logistic regression trained on synthetic set | Accuracy, per-class F1 |
| Text extraction | PDF text layer when present; else PaddleOCR/Tesseract | Azure Document Intelligence | Character error rate on a sample |
| Field extraction | Regex + anchor keywords + table parsing | LLM with JSON schema for stubborn fields | Field-level exact-match and fuzzy-match F1 |
| Cross-document match | Normalised string + fuzzy (RapidFuzz) | (none needed) | Detection rate of injected mismatches; false-positive rate |
| Tamper triage | Rule signals: incremental-update count, date gap, producer mismatch, font anomalies, text-layer vs OCR diff, duplicate hash | ELA on embedded images; copy-move check | Per-indicator precision/recall on synthetic injections |
| Infrastructure advice | EXIF checks, duplicate/perceptual-hash, structured area/room cross-check | Zero-shot image tagging (CLIP) for scene type | Agreement with labels; always advisory |

## 12.3 Evaluation

| PS evaluation item | How we measure | Honest caveat |
|---|---|---|
| Document-extraction accuracy | Field-level accuracy on held-out synthetic documents | Synthetic is easier than real; say so |
| Validation accuracy | Agreement between system findings and ground-truth defect labels | Same |
| Anomaly-detection performance | Precision/recall per indicator on injected edits; false positives on clean docs | Not real-world forgery performance |

| PS evaluation item | How we measure | Honest caveat |
|---|---|---|
| Successful source-verification rate | Share of documents with a conclusive state under normal mock; and routing correctness under injected failures | Mock sources — the number is about our workflow |
| Average verification time | Median end-to-end seconds per application on the demo host | Report hardware |
| Reduction in manual effort | Do NOT claim a percentage. Run a timed comparison with teammates (manual checklist vs tool-assisted) and report n and method | Tiny sample; label it illustrative |

**Rule for every number in the PPT.** Only quote a metric you computed with the script in eval/, state the data (synthetic, n = ?), and keep the script in the repo. Never quote accuracy you have not measured.

# 13. 8-Day Sprint Roadmap

| Day | Theme | Deliverable (end of day) | Owner |
|---|---|---|---|
| 1 | Foundation | Supabase up, tables created, RLS, Clerk configured, Next.js boilerplate with 3-role auth, GitHub repo with CI | Full-stack lead + DB lead |
| 2 | Upload + Queue + OCR baseline | File upload to private bucket, sha256 computed, job row inserted, worker polls and calls FastAPI /classify + /ocr+extract on one synthetic PDF | Full-stack lead + AI/Python lead |
| 3 | Rule engine + findings | TypeScript rule evaluator with 6 rule types, unit tests passing, EOA_EXISTING.v1.json, findings saved with evidence (page, bbox), institution report page | Rule engine lead |
| 4 | Verification adapters + tamper triage | Mock issuer API, mock AICTE records API, 7-state state machine, /tamper with 4 indicators, officer console showing findings with confirm/dismiss | AI/Python lead + integration lead |
| 5 | Dashboard + audit + alerts | Officer work queue ranked by priority, audit log with hash-chain verification, in-app alerts for 5 alert types, metrics page with eval numbers | Full-stack lead + data/eval lead |
| 6 | Demo path + polish | Full end-to-end demo script rehearsed twice, all P0 features working, fallback tested (AI service killed + recovered) | All |
| 7 | PPT + video | 6-slide deck finished, 2-minute demo video recorded, README updated, eval script run and numbers pasted into PPT | PPT lead + data/eval lead |
| 8 | Buffer + submission | Team review, final checks on portal requirements, upload by 18:00 on 27 October | Team lead |

## 13.1 P0 definition of done

- Institution can log in, upload 5 synthetic documents, and receive a deficiency report with at least 3 evidence-linked findings.
- Officer can confirm and dismiss findings; each action is in the audit log.
- Verification adapter produces all 7 states on the synthetic set (including a fault-injected SOURCE_UNAVAILABLE that retries and routes to MANUAL_REVIEW_REQUIRED).
- Audit chain verify-chain button reports "OK" and then "broken at row N" after a deliberate DB edit.
- All metrics in Section 12.3 are computed by the eval script and pasted into the metrics page.

# 14. Six-Slide SIH Idea Submission Deck

Use these exact slide titles and content. The SIH template specifies exactly six slides. Every claim must be supportable.

## Slide 1 — Title Page

| Field | Content |
|---|---|
| Title | NirikshAI: AI-Powered Regulatory Compliance and Document Authentication for AICTE EoA Approvals |
| Subtitle | "Check early. Prove everything. Humans decide." |
| Team name | [Your team name] |
| Institute | [Institute name, city, state] |
| Problem Statement ID | SIH 2026 — PS 26261 |
| Organization | AICTE, Regulation Bureau, Ministry of Education, Government of India |
| Date | 28 October 2026 |

## Slide 2 — Proposed Solution

- **Problem:** AICTE scrutiny officers manually check hundreds of EoA applications for missing, expired or forged documents — slow, inconsistent, legally untracked.
- **NirikshAI:** (1) classifies each application and loads the configurable Handbook checklist, (2) reads every document via OCR and extracts typed fields, (3) checks fields against each other and against AICTE institutional records, (4) triages documents for tampering indicators (never accusations), (5) tracks issuer verification in seven explicit states, and (6) writes every step to a hash-chained audit log.
- **Innovation 1 — Evidence-linked findings:** every flag cites the exact document, page and bounding box.
- **Innovation 2 — Rules as versioned data:** checklists are JSON rule sets that cite AICTE Handbook clauses. Policy cell staff update rules without touching code.
- **Innovation 3 — Seven-state verification with fail-safe routing:** source unavailable retries with back-off; it never becomes a silent pass.
- **Innovation 4 — Tamper triage without overclaiming:** layered forensic signals produce "review" or "priority review" labels. The word "forged" never appears.
- **Innovation 5 — Hash-chained audit log:** each row stores SHA-256(prev_hash + canonical event JSON). Any row edit is detected instantly. No blockchain required.

## Slides 3–5 — Technical Approach, Feasibility, Impact

See the corresponding sections in this report (Sections 9–12) for the full technical content, and Sections 4–5 for the gap analysis and scoring. Map each claim to the section and slide template heading exactly as the SIH reviewer instructions specify.

## Slide 6 — Research and References

- AICTE Approval Process Handbook 2024-27 [Official] — primary source for rule clauses (Day-1 task)

• DigiLocker API specification — api.apisetu.gov.in [Verified]
• EoA letters (public institution copies) — cvru.ac.in, nbs.edu.in [Verified; not redistributed]
• SIH 2026 guidelines — whereuelevate.com, blogs.reskilll.com [Verified, secondary]
• PyMuPDF, pikepdf, pyHanko, PaddleOCR, Tesseract, imagehash — open-source library docs [Official]
• Supabase documentation — supabase.com/docs [Official]
• Clerk documentation — clerk.com/docs [Official]

# 15. Team Roles and Responsibility Matrix

SIH 2026 requires exactly 6 team members from the same institute, with at least 1 female member [Verified, secondary]. Assign specific names to roles below.

| Role | Responsibilities | Primary deliverables | Member |
|---|---|---|---|
| Team Lead / Integration Lead | Daily standups, end-to-end integration, demo script, submission upload | Working demo, README, submission checklist | _____ |
| Full-Stack Lead (Next.js) | Institution portal, officer console, admin rule manager, Supabase integration, Clerk auth, Realtime | All Next.js pages, API routes, RLS policies | _____ |
| AI / Python Lead (FastAPI) | /classify, /ocr+extract, /tamper, /infra-advice endpoints; Docker build | AI service Docker image, pytest suite | _____ |
| Rule Engine Lead (TypeScript) | Evaluator (~300 LOC), 6 rule types, EOA_EXISTING.v1.json, unit tests | evaluator.ts, evaluator.test.ts, rule set JSON | _____ |
| Data / Evaluation Lead | Synthetic corpus generation (150-200 docs), labels.csv, eval scripts, metrics page numbers | data/synthetic/, eval/evaluate_metrics.py | _____ |
| PPT / Documentation Lead | 6-slide deck, demo video, architecture diagrams, this report | NirikshAI_Slides.pdf, demo video, nirikshai_report.pdf | _____ |

# 16. Reviewer Q&A; Defense and Viva Preparation

Prepare every team member to answer these 25 questions. Admitting limits builds credibility.

## 16.1 System design questions

### Q1: Why not use a large language model (LLM) to check all compliance at once?

Rule decisions must be explainable and auditable per the PS safeguards. An LLM cannot cite a Handbook clause, version its decision, or guarantee a deterministic output. We use LLMs optionally for OCR post-processing of messy fields, with JSON schema validation of the output. Rule outcomes are always deterministic TypeScript.

### Q2: How does the rule engine handle a new AICTE Handbook cycle?

An admin publishes a new rule set version (EOA_EXISTING.v2.json) through the rule manager or by uploading the JSON file. The old version is retained. Every finding records the rule_set_id and version it was evaluated against, so historic findings are still explainable.

### Q3: What happens if the OCR gets a field wrong?

The finding carries the confidence score. Low-confidence extractions are flagged as "uncertain — please verify" rather than generating a definitive finding. The officer can read the highlighted region in the PDF viewer and override the extracted value; the override is an audit event.

### Q4: How does the hash-chained audit log work, and can't a DBA break it?

hash = SHA-256(prev_hash || canonical_json(row)). Any row edit changes that row's hash, which breaks the chain from that point forward. The verify-chain button detects this. A sufficiently privileged DBA could rewrite the entire chain. The countermeasure — periodic external anchoring of the latest hash — is documented as future scope.

### Q5: How is the system integrated with the AICTE portal?

The prototype is not integrated with the live AICTE portal; we used mock APIs. We designed and documented a REST contract (Section 10.6) that the AICTE portal team could implement to give us read-only access to institutional records and to receive our validation results. The integration design is integration-ready, not integration-complete.

## 16.2 AI and forensics questions

### Q6: Can your tamper detection be fooled?

Yes, by a sophisticated attacker who understands our signals. Our position: these are "preliminary indicators requiring human verification" as the PS states. We explicitly avoid the word "forged". The value is that an officer gets a prioritised shortlist; they still make the judgment.

### Q7: What is your OCR accuracy on real documents?

We have not tested on real documents. On our labelled synthetic corpus (n = ___), field-level accuracy is ___%. Synthetic documents are cleaner than real scans; real accuracy will be lower. We report the metric with sample size, hardware, and the caveat that it is on synthetic data.

### Q8: Why PaddleOCR and not Azure or Google?

PaddleOCR is free, runs on our server, and keeps documents inside our boundary — important for institutional data. Azure Document Intelligence is integrated behind an interface as an optional upgrade; switching is a single environment variable change.

### Q9: Can you actually verify documents with real issuing authorities?

Not in the prototype; we used a mock issuer service. We built a pluggable adapter interface: any issuer with an API, a signed URL, or a QR code can be added as a new adapter. DigiLocker integration would require AICTE to become a registered requester organisation — a six-step process described in the DigiLocker API documentation.

### Q10: How do you handle regional-language documents?

PaddleOCR supports Devanagari (Hindi). For the prototype, we tested only English. Hindi support is a P2 feature: the OCR engine supports it; the field-extraction regex patterns need to be extended for Devanagari text.

## 16.3 Security and compliance questions

### Q11: How do you prevent an institution from accessing another institution's documents?

Supabase Row Level Security on every table. Institution users have a Clerk JWT that contains their institution_id. RLS policies allow SELECT/INSERT only on rows where institution_id matches. The service-role key (which bypasses RLS) is used only on the server, never in the browser.

### Q12: What if a tamper indicator flags a legitimate document?

The finding status is "open"; the officer confirms or dismisses it with a written reason. Dismissal is an audit event. Dismissed findings feed back into a false-flag metric we track on the metrics page.

### Q13: How do you handle the DPDP Act 2023 and data retention?

Data classes are defined with configurable retention periods (default 7 years for audit logs, shorter for raw document files). A scheduled purge job deletes files past their retention date. Personal identifiers are masked in the UI — last 4 digits only. The policy numbers must be confirmed with AICTE's legal team.

### Q14: What is your threat model?

Four threats explicitly addressed: (1) Malicious upload — sandboxed parsing with timeout, file-type and size limits, PDF active-content stripping. (2) IDOR — RLS ensures you cannot access another institution's data even if you know the UUID. (3) Log tampering — hash-chain; detect any row edit. (4) Adversarial documents designed to fool OCR — low-confidence extractions are flagged, not silently passed.

### Q15: Are any decisions automated without human review?

No. The data model has no "auto-reject" or "auto-approve" state. Every adverse outcome requires an officer action with a decision field and notes field. This is enforced at the schema level, not just the UI level.

## 16.4 Business and scalability questions

### Q16: How does this scale to AICTE's actual volume of applications?

For production: (1) Replace the Postgres job queue with a proper message broker for fan-out. (2) Scale the AI service horizontally behind a load balancer. (3) Partition the database by region/cycle. (4) The rule engine is stateless and parallelisable. The two-deployable prototype demonstrates the design; the scaling plan is documented.

### Q17: What is the business model / who pays for this?

This is a government regulatory tool, not a commercial product. The natural home is AICTE's own infrastructure budget. Cloud costs at prototype scale: Vercel free tier (Next.js), Supabase free tier (Postgres + Storage), and one container instance (Render free tier or Azure student credit).

### Q18: What happens if AICTE changes the Approval Process Handbook mid-cycle?

Admin publishes a new rule-set version. Applications already in review continue against the rule set they were submitted under. New applications use the new version. No code deployment required.

### Q19: What would it take to go from prototype to production?

Four things not in the prototype: (1) Real issuer credentials and integration agreements. (2) Read-only access to AICTE institutional records via the contract we defined. (3) Security audit and AICTE data protection clearance. (4) A parallel pilot with real applications to calibrate rule thresholds and OCR confidence cuts.

### Q20: Why not build this as an extension of the existing AICTE portal?

The PS says "integrated with, or designed for integration with, the AICTE portal". We chose "designed for integration": (a) we do not have access to the portal's codebase; (b) a sidecar approach means AICTE can pilot NirikshAI without touching the live portal; (c) the REST contract we defined allows the portal to call NirikshAI as a service.

## 16.5 Evaluation and metrics questions

### Q21: What is your validation accuracy?

On the synthetic labelled corpus (n = ___, ___% tampered, ___% mismatched, ___% clean), the rule engine correctly flagged ___% of injected defects with zero false positives on clean documents. We will fill in these numbers when the eval script finishes running. We state the method, the sample, and the caveats (synthetic is easier than real).

### Q22: How do you measure "reduction in manual effort"?

We ran a timed study with ___ teammates: each checked the same 5 synthetic applications manually (using a printed checklist) and then with the tool. We report the median time per application in each condition, n, and the note that this is an illustrative internal pilot, not a formal study.

### Q23: How did you evaluate the tamper indicators specifically?

We injected defects into the synthetic corpus: changed a date in a PDF with a text editor (creates an incremental update), duplicated a page from another application, changed a font mid-document. The /tamper endpoint output was compared to the ground-truth labels.csv. We report per-indicator precision and recall.

### Q24: Are your evaluation metrics comparable to any published benchmark?

No directly comparable benchmark exists for AICTE EoA document validation. We are explicit that this is a first measurement on synthetic data and that real-world performance requires a pilot with real documents.

## Q25: What would you do differently with another month?

Three things: (1) Obtain a sample of real (anonymised) AICTE EoA applications and retune OCR confidence thresholds and rule parameters. (2) Build the real DigiLocker adapter (requires AICTE credentials). (3) Replace the Postgres job queue with a proper message broker and run a load test.

# 17. Risk Assessment and Contingency Matrix

| Risk | Prob | Impact | Mitigation | Contingency |
|---|---|---|---|---|
| OCR low accuracy on scanned documents | High | Med | PaddleOCR + preprocessing; Azure DI as upgrade | Flag low-confidence fields; officer views raw PDF; state honestly in PPT |
| Supabase free tier pauses | Med | High | Check activity daily; keep docker-compose ready | Un-pause and run local for the demo |
| Azure credit not available by Day 3 | Med | Low | Use Render or Fly.io free tier instead | Run AI service locally on a laptop |
| Team member unavailable | Low | High | Each P0 feature has two people who understand it | Reassign; cut P1 features |
| AI service cold start for demo | Med | Med | Keep-alive ping every 5 minutes; pre-warm before demo | Run service locally |
| SIH portal submission issues | Low | High | Submit 24 hours early (27 October evening) | Email SPOC with proof of attempt |
| Jury asks for real issuer integration | High | Low | Prepare Q9 answer; show adapter contract and mock | Demonstrate mock issuer live; explain DigiLocker credential path |
| Clerk or Supabase Auth outage during demo | Low | High | DEMO_MODE=true with 3 seeded users (clearly labelled on screen) | Switch to DEMO_MODE; continue demo |

# 18. References, Standards and Citations

## 18.1 Primary sources (official)

- **AICTE Approval Process Handbook 2024-27** — Governs all EoA processes. Download from aicte-india.org. [Status: must read on Day 1; all rule clause IDs in the prototype are placeholders].
- **DPDP Act 2023** — Digital Personal Data Protection Act 2023, Government of India. Governs data retention, consent and privacy obligations.
- **DigiLocker API documentation** — api.apisetu.gov.in. Specifies the PKCE OAuth2 flow, document retrieval, and HMAC signature verification. [Verified].
- **SIH 2026 problem statement text** — PS 26261, AICTE Regulation Bureau. Source for all [Official PS] tags in this report. [Provided by the user].

## 18.2 Verified secondary sources

- SIH 2026 competition overview — whereuelevate.com, blogs.reskilll.com [Verified, secondary; retrieved October 2026].
- SIH team eligibility and submission format — blogs.reskilll.com, bcrec.ac.in [Verified, secondary].
- Supabase free-tier limits (500 MB DB, 1 GB storage, 7-day inactivity pause) — supabase.com/pricing; secondary confirmations at focusreactive.com, automationatlas.io [Verified].
- EoA letter format — publicly published PDFs at cvru.ac.in, nbs.edu.in, abit.edu.in [Verified; not redistributed; used to understand format only].
- Kerala DTE supporting document list — dtekerala.gov.in circular [Verified].
- DigiLocker integration path summary — didit.me [Verified, secondary].

## 18.3 Libraries and technical references

- PyMuPDF (fitz) — pymupdf.readthedocs.io — PDF parsing, text extraction, image extraction.
- pikepdf — pikepdf.readthedocs.io — PDF incremental-update analysis, metadata.
- pyHanko — pyhanko.readthedocs.io — PDF digital signature verification.
- PaddleOCR — github.com/PaddlePaddle/PaddleOCR — multilingual OCR engine.
- Tesseract OCR — github.com/tesseract-ocr — fallback OCR engine.
- imagehash — github.com/JohannesBuchner/imagehash — perceptual hash for duplicate detection.
- OpenCV — opencv.org — image preprocessing and error-level analysis.
- RapidFuzz — github.com/rapidfuzz/RapidFuzz — fast fuzzy string matching.
- FastAPI — fastapi.tiangolo.com — Python async API framework.
- Next.js — nextjs.org/docs — React framework with App Router.
- Supabase — supabase.com/docs — Postgres-as-a-service with RLS and Storage.
- Clerk — clerk.com/docs — Authentication and role management.
- ReportLab — reportlab.com/docs — Python PDF generation library (used for this report).

# 19. Appendix: Complete Schemas, Rule Definitions and Verification Contracts

## 19.1 Complete rule type registry

| Rule type | Parameters | Logic | Finding kind if failed |
|---|---|---|---|
| required_document | doc_type: string | At least one document of this type exists with analysis_status != failed | missing |
| field_present | doc_type, field_key | extracted_fields contains a row for this doc + key with value != null | missing |
| date_valid_for_year | doc_type, field_key, year: "YYYY-YY" | Parsed date > 31 March of the second year in the academic year string | expired |
| cross_field_equal | fields: [doc_type.field_key, ...], fuzzy: 0-1 | All field values match with similarity >= fuzzy after normalisation | inconsistent |
| numeric_leq | doc_type, field_key, max_value: number | Parsed numeric value <= max_value | inconsistent |
| record_equals | field: doc_type.field_key, record: string, fuzzy: 0-1 | Extracted field matches institution_records value with similarity >= fuzzy | record_mismatch |
| status_equals | record: string, value: string | institution_records[record] === value (exact, case-normalised) | record_mismatch |
| identifier_format | doc_type, field_key, regex: string | Extracted value matches the regex pattern | inconsistent |

## 19.2 Finding evidence schema (TypeScript)

```
interface Finding {
  id:           string;          // uuid
  application_id: string;
  rule_set_id:  string;          // which version evaluated this
  rule_id:      string;          // e.g. "R-EXP-NOC"
  kind:         FindingKind;     // missing | expired | inconsistent | ...
  severity:     "blocking" | "warning" | "info";
  message:      string;          // plain English for the officer
  evidence: {
    doc_ids:    string[];        // document UUIDs involved
    pages:      number[];        // 0-indexed page numbers
    bboxes:     BBox[];          // {x, y, w, h} normalised 0-1
    field_key:  string;
    extracted_value: string | null;
    expected_value:  string | null;
    confidence: number | null;   // OCR confidence 0-1
    clause:     string;          // e.g. "APH 2024-27 §4.2.1"
  };
  status: "open" | "confirmed" | "dismissed";
  created_at: string;
}
```

## 19.3 Verification adapter interface (TypeScript)

```
type VerifStatus =
  | "verified"
  | "mismatch_detected"
  | "expired"
  | "verification_pending"
  | "source_unavailable"
  | "manual_review_required";

interface VerifResult {
  status:      VerifStatus;
  source:      string;           // "mock_issuer" | "digilocker" | "qr_lookup" | "manual"
  timestamp:   string;           // ISO 8601
  request_ref: string | null;
  comparison: {
    field_key:       string;
    submitted_value: string;
    source_value:    string | null;
    match:           boolean | null;
  }[];
}

interface VerificationAdapter {
  supports(doc_type: string): boolean;
  verify(
    doc: Document,
    fields: ExtractedField[],
    options?: { timeout_ms?: number }
  ): Promise<VerifResult>;
}
```

## 19.4 docker-compose.yml (offline fallback)

```
version: "3.9"
services:
  postgres:
    image: postgres:16
    environment: { POSTGRES_PASSWORD: demo, POSTGRES_DB: nirikshai }
    ports: ["5432:5432"]
    volumes: ["./db/schema.sql:/docker-entrypoint-initdb.d/01_schema.sql"]

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment: { MINIO_ROOT_USER: minioadmin, MINIO_ROOT_PASSWORD: minioadmin }
    ports: ["9000:9000", "9001:9001"]

  ai_service:
    build: ./services/ai
    environment: { OCR_ENGINE: tesseract }
    ports: ["8001:8001"]

  mock_issuer:
    build: ./services/mocks
    environment: { SERVICE: issuer }
    ports: ["8002:8002"]

  mock_records:
    build: ./services/mocks
    environment: { SERVICE: records }
    ports: ["8003:8003"]
```

**End of NirikshAI Technical Report — SIH 2026 Problem Statement 26261**

*Prepared 20 October 2026. All code, metrics and claims described are plans as of this date; nothing has been deployed or tested against real data.*